

EXERCÍCIOS

Nome: Jaydim Benício RabeloMatrícula: R5 97 717

Prof: MSc. Mateus de Paula

Data: 21/09/2026

1. O que caracteriza uma classe e um objeto no contexto de POO?
2. Explique o conceito de encapsulamento e qual é a convenção utilizada em Python para indicar que um atributo ou método deve ser tratado como privado.
3. Como funciona o princípio da herança entre classes e de que forma uma subclasse pode estender o comportamento de uma superclasse?
4. O que é o polimorfismo e como ele se manifesta na relação entre subclasses e superclasses?
5. Qual é a função do primeiro argumento (geralmente chamado de self) em um método de instância em Python?
6. Explique a diferença prática e conceitual entre um método de instância, um método estático (@staticmethod) e um método de classe.
7. O que são atributos de dados de classe (class data attributes) e como eles diferem dos atributos de instância em termos de compartilhamento de memória?
8. No contexto do padrão de projeto Template Method, o que são métodos "hook" e como eles ajudam a separar o processamento genérico de regras de negócio específicas?
9. De acordo com a definição de Clean Code apresentada por Robert C. Martin (Uncle Bob), qual é o objetivo principal dessa filosofia de desenvolvimento?
10. Como o inventor do C++, Bjarne Stroustrup, descreve a relação entre um código limpo, eficiência e a quantidade de tarefas que uma rotina deve executar?
11. Ao definir nomes significativos no código, quais são as três perguntas fundamentais que o nome escolhido deve responder?
12. Explique por que, segundo as boas práticas de Clean Code, os desenvolvedores não devem ter medo de utilizar nomes grandes para variáveis ou métodos.
13. Com base na convenção apresentada para a Programação Orientada a Objetos, como devem ser representadas as classes e como devem ser representados os métodos?
14. O slide menciona que "métodos com muitos níveis implicam em muita responsabilidade". Como o princípio de "fazer uma única coisa" ajuda a resolver esse problema?
15. O que é um Code Smell e de que forma um código ruim pode impactar o andamento de um projeto e a carreira do desenvolvedor a longo prazo?
16. Explique o conceito do princípio DRY (Don't Repeat Yourself) e qual prática comum entre programadores ele tenta evitar.
17. Por que o uso excessivo de comentários é visto com ressalvas no Clean Code e qual é a alternativa recomendada para tornar o código auto documentado?
18. Quais são as cinco regras descritas pelo anacrônico FIRST para a criação de testes limpos?
19. Como o Princípio da Responsabilidade Única (SRP) define o motivo para uma classe ser modificada e como a falta dele afeta a coesão na POO?
20. O Princípio do Aberto/Fechado (OCP) afirma que entidades devem estar "abertas para extensão, mas fechadas para modificação". Como a herança e o polimorfismo da POO ajudam a aplicar esse conceito?
21. Explique o Princípio de Substituição de Liskov (LSP) e o problema de abstração que ocorre quando uma classe derivada necessita de "baterias" (recursos extras não previstos na classe base).
22. Considerando que o Clean Code foca na clareza de variáveis, métodos e classes, de que maneira os conceitos do SOLID apresentados (como o SRP e o DIP) servem como base para a Arquitetura Limpa na organização dos módulos, regras de negócio e dependências externas de um sistema?

respostas:

- 1) Uma classe é como um molde abstrato definido por características e comportamentos esperados. O objeto é a instância física e concreta dessa classe rodando na memória da aplicação.
- 2) Consiste em proteger os dados internos de uma classe, controlando e restringindo seu acesso pelo exterior. Em Python, a convenção é `"_"`.
- 3) Permite que novas classes absorvam atributos e métodos de uma classe base já existente, promovendo o reuso. Subclasses estendem esse comportamento ao adicionar rotinas novas ou sobrescrever lógicas da superclasse.
- 4) É a capacidade de tratar objetos de diferentes subclasses através de uma mesma interface de classe pai. Manifesta-se quando a mesma chamada de método executa comportamentos diferentes dependendo da subclasse instanciada.
- 5) Atua em python como uma referência explícita à própria instância do objeto que o método é chamado. Garante que o método acesse e modifique os dados em memória específicos daquele objeto.
- 6) Tipos de métodos são: métodos de instância que alteram o estado do objeto; os de classe atuam no estado global da própria classe. Os estáticos não acessam nem instância nem classe, operando isoladamente como funções utilitárias.

7) São variáveis globais da classe e compartilham exatamente o mesmo espaço em memória para todas as instâncias. Atributos de instância são individuais, com espaços de memória separados para cada objeto criado.

8) São operações padrões deixadas na superclasse para serem opcionalmente substituídas pelas subclasses. Garantem que o esqueleto do algoritmo se mantenha intacto enquanto as regras de negócio variam nos ganchos.

9) Foca primariamente em produzir código extremamente fácil de ser lido, compreendido, para uma melhor revisão.

10) Ele defende que código limpo faz apenas uma coisa bem feita, focando em elegância e performance. Na lógica muito direta, impede que bugs consigam se esconder no meio de rotinas complexas.

11) A nomenclatura deve responder claramente por que existe, o que efetivamente faz e como é utilizada. Se a escolha do nome exigir explicações extras, ele falhou no seu propósito principal.

12) Um nome longo e perfeitamente descritivo é muito superior a variáveis curtas, genéricas e enigmáticas. Não se deve temer a verbosidade se ela eliminar ambiguidades e a necessidade de comentários explicativos.

13) As classes costumam ser representadas por substantivos, definindo as entidades do sistema. Já os métodos devem ser representados por verbos que traduzem as ações que a classe realiza.

14) Fazer apenas uma coisa evita o acúmulo de responsabilidades e múltiplos níveis de aninhamento de código. Isso divide processos complexos e funções menores e coisas, facilitando a leitura e a manutenção.

15) São indícios claros de design ruim os atalhos mal feitos na base do código. A longo prazo, acumulam débito técnico insustentável, inibem melhorias e desmotivam completamente o desenvolvedor.

16) Exige que não haja duplicação de lógica ou conhecimento de negócio espalhado ao longo do sistema. Ele lida com o vício de "copiar e colar", que gera inconsistências severas durante manutenções futuras.

17) O uso excessivo costuma mascarar códigos confusos e os comentários rapidamente se tornam mentirosos por atualização. A alternativa primária é tornar o código auto-documentado através de nomenclaturas impecáveis.

18) Para serem limpos, os testes devem ser rápidos, independentes, repetíveis, auto-analisáveis e oportunos.

19) Uma classe deve ter apenas um motivo para ser alterada, limitando rigorosamente seu escopo funcional. A ausência SRP destrói a coesão criando classes massivas difíceis de testar.

20) A herança e o polimorfismo permitem adicionar funcionalidades criando novas subclasses independentes. Estende-se

o comportamento (aberto) sem tocar ou arriscar o código da superclasse já consolidada (fechado).

21) Uma subclasse deve sempre substituir sua superclasse sem quebrar as expectativas originais do programa. O problema de "baterias" surge quando a derivada requer gambiarras ou regras que a base desconhecia.

22) O SRP escala garantindo que diferentes módulos tenham estritamente de responsabilidades únicas e isoladas. Já a Inversão de dependência, protege o núcleo de negócios de interfaces externas (BD, UI) via abstrações puras.